

Rapport du Projet — Portefeuille Investissement

Réalisé par:

Farah FASSI
Chaima CHATTI

1. Présentation du projet

Ce projet est une application web complète de gestion de portefeuille d'investissement. Elle permet à des investisseurs de suivre leurs actifs financiers, d'enregistrer leurs transactions d'achat et de vente, de gérer leur budget, et d'analyser leurs performances. Une interface administrateur permet à un administrateur de gérer les actifs disponibles sur la plateforme.

Technologies utilisées

- Framework : ASP.NET Core 10 avec Blazor Server
- ORM : Entity Framework Core 10
- Base de données : SQLite (fichier portefeuille.db)
- Authentification : ASP.NET Core Identity
- Interface : Bootstrap 5 + Radzen Blazor
- Langage : C# 13

2. Architecture de l'application

L'application suit une architecture en couches bien délimitées, ce qui facilite la maintenance et l'évolution du code.

Couche Modèles (Models/)

Cette couche contient toutes les entités du domaine métier. Chaque classe représente un concept réel du système.

- `ApplicationUser` : l'utilisateur de base, il hérite de `IdentityUser` pour bénéficier du système d'authentification d'ASP.NET Core Identity (email, mot de passe haché, rôles).
- `Investisseur` : le profil financier d'un utilisateur. Il contient le budget total, le budget disponible et les points de fidélité. Il est lié à `ApplicationUser` par une relation 1-à-1.
- `Administrateur` : le profil d'un utilisateur ayant les droits de gestion des actifs et des utilisateurs.
- `Actif` : représente un instrument financier (action, crypto-monnaie, ETF ou obligation). Contient le prix actuel, le prix d'achat moyen et la quantité disponible.
- `Transaction` : une opération d'achat ou de vente sur un actif. Elle est liée à un investisseur et à un actif.
- `BudgetInvestissement` : l'historique des allocations de budget effectuées par un investisseur.
- `PeriodeInvestissement` : une période optionnelle à laquelle une transaction peut être rattachée.

Couche Données (Data/)

`AppDbContext` est le contexte Entity Framework Core. Il hérite de `IdentityDbContext`, ce qui crée automatiquement les tables nécessaires à l'authentification. La méthode `OnModelCreating` configure toutes les relations entre entités : les relations 1-à-1 entre `ApplicationUser` et ses profils, les relations 1-à-plusieurs entre `Investisseur` et ses transactions, et les comportements de suppression en cascade.

Couche Services (Services/)

Chaque service est défini par une interface et une implémentation concrète. Ce patron de conception permet de découpler les composants et de faciliter les tests unitaires.

- `IActifService / ActifService` : gère toutes les opérations sur les actifs (création, modification, suppression, recherche).
- `ITransactionService / TransactionService` : gère les transactions et fournit les calculs statistiques (valeur du portefeuille, gain/perte global, performance par actif, répartition par type).

- InvestisseurService / InvestisseurService : gère les profils investisseurs et les opérations sur le budget.

Couche Présentation (Components/)

Les pages Blazor se trouvent dans Components/Pages/. Les composants réutilisables sont dans Components/UI/. La mise en page globale (menu latéral et barre supérieure) est dans Components/Layout/.

3. Modèle de données et relations

La base de données SQLite contient les tables suivantes, générées par Entity Framework Core à partir des modèles C#.

Relations principales

- ApplicationUser — Investisseur : relation 1-à-1. Un utilisateur peut avoir un seul profil investisseur. La clé étrangère UserId dans la table Investisseurs référence l'identifiant de l'utilisateur.
- ApplicationUser — Administrateur : relation 1-à-1, même principe.
- Investisseur — Transaction : relation 1-à-plusieurs. Un investisseur peut avoir plusieurs transactions. La suppression d'un investisseur entraîne la suppression de toutes ses transactions (Cascade Delete).
- Actif — Transaction : relation 1-à-plusieurs. Un actif peut apparaître dans plusieurs transactions.
- PeriodeInvestissement — Transaction : relation optionnelle. Si une période est supprimée, le champ Periodeld des transactions devient NULL (SetNull).
- Investisseur — BudgetInvestissement : relation 1-à-plusieurs pour garder l'historique de toutes les allocations budgétaires.

Propriétés calculées

Deux propriétés de la classe Actif sont marquées [NotMapped] : QuantiteTotaleDetenue et ValeurActuellePosition. Ces propriétés sont calculées en mémoire à partir de la liste des transactions et ne sont pas stockées en base de données. Cela évite les redondances et garantit la cohérence des données.

4. Authentification et gestion des rôles

L'application utilise ASP.NET Core Identity pour l'authentification. Deux rôles sont définis : Administrateur et Investisseur. Ces rôles sont créés automatiquement au démarrage de l'application si ils n'existent pas.

Flux d'inscription

Lorsqu'un utilisateur s'inscrit via le formulaire /signup, un endpoint Minimal API reçoit les données du formulaire. Il vérifie que les mots de passe correspondent et que l'email n'est pas déjà utilisé. Si tout est valide, il crée un ApplicationUser via UserManager, lui assigne le rôle Investisseur, puis crée un profil Investisseur avec le budget initial saisi. L'utilisateur est ensuite redirigé vers la page de connexion.

Flux de connexion

La connexion se fait via l'endpoint /api/auth/login. SignInManager vérifie les identifiants et crée le cookie de session en cas de succès. L'utilisateur est redirigé vers le tableau de bord. En cas d'échec, un message d'erreur est passé en paramètre de l'URL.

Protection des pages

Chaque page protégée porte l'attribut [Authorize]. Dans les composants, la vérification du rôle se fait via `user.IsInRole("Administrateur")`. Le menu latéral utilise le composant `AuthorizeView` avec `Roles="Administrateur"` pour n'afficher la section administration qu'aux administrateurs.

5. Fonctionnalités de l'application

Tableau de bord (Dashboard)

La page d'accueil après connexion affiche une vue synthétique du portefeuille : la valeur totale actuelle, le gain ou la perte global en valeur et en pourcentage, le budget disponible, le nombre de transactions. Elle affiche également la répartition des investissements par type d'actif (actions, crypto, ETF, obligations) sous forme de barres de progression, ainsi que la performance détaillée de chaque position et les cinq dernières transactions.

Gestion des actifs

La page /actifs affiche tous les actifs disponibles sous forme de cartes. Un investisseur peut filtrer par nom, symbole ou type, puis acheter ou vendre un actif via un formulaire modal. Un administrateur peut en plus créer de nouveaux actifs, modifier les existants, et les supprimer. Toutes les actions d'administration sont protégées par la vérification du rôle au niveau du composant.

Transactions

La page /transactions liste toutes les transactions de l'investisseur connecté avec la possibilité de filtrer par type (achat/vente), statut (confirmée, en attente, annulée), et symbole d'actif. L'investisseur peut supprimer une transaction après confirmation.

Budget

La page /budget permet à un investisseur de visualiser son budget total, son budget disponible, le montant investi, et l'historique de toutes ses allocations. Il peut également ajouter un montant à son budget. Pour un administrateur, cette page affiche les budgets de tous les investisseurs dans un tableau récapitulatif.

Analytique

La page /analytics fournit une analyse approfondie du portefeuille : indicateurs de performance clés, graphes de répartition par type d'actif, détail des gains et pertes par position. Elle identifie automatiquement la meilleure et la pire performance du portefeuille.

6. Choix techniques et justifications

Pourquoi Blazor Server ?

Blazor Server a été choisi car il permet d'accéder directement à la base de données depuis le serveur, sans avoir besoin d'une API REST séparée. La connexion SignalR maintient un canal de communication permanent entre le navigateur et le serveur, ce qui rend l'interface réactive. De plus, les données sensibles (transactions, budgets) ne transitent pas en clair dans un bundle téléchargeable, contrairement à Blazor WebAssembly.

Pourquoi le pattern Interface/Implémentation pour les services ?

Chaque service est défini par une interface (IActifService) et une classe concrète (ActifService). Ce choix respecte les principes SOLID, notamment l'inversion de dépendances. Les composants Blazor ne dépendent que de l'interface, ce qui facilite les tests unitaires (on peut injecter un mock) et permet de changer l'implémentation sans modifier les composants.

Pourquoi les services en mode Scoped ?

Les services sont enregistrés avec AddScoped dans le conteneur d'injection de dépendances. Cela signifie qu'une instance est créée par connexion SignalR (par circuit Blazor). C'est le cycle de vie recommandé car ApplicationDbContext, qui gère les transactions de base de données, est lui-même Scoped. Utiliser Singleton provoquerait des conflits entre utilisateurs simultanés.

Seed automatique

Au démarrage, l'application crée automatiquement les rôles Administrateur et Investisseur s'ils n'existent pas, ainsi qu'un compte administrateur par défaut. Cela garantit qu'un premier administrateur peut toujours se connecter sans intervention manuelle sur la base de données.

7. Limites identifiées et améliorations possibles

- Précision financière : les montants sont stockés en type float. En production, le type decimal serait plus approprié car il évite les imprécisions de représentation binaire en virgule flottante.

- Concurrence : si deux investisseurs achètent simultanément le même actif, des conditions de course peuvent survenir sur la mise à jour du budget. Une solution serait d'utiliser des transactions de base de données explicites avec un niveau d'isolation adapté.
- Prix en temps réel : les prix des actifs sont saisis manuellement par l'administrateur. Une intégration avec une API financière externe permettrait de les mettre à jour automatiquement.
- Pagination : les listes de transactions et d'actifs ne sont pas paginées, ce qui peut poser des problèmes de performance pour de grands volumes de données.
- Tests unitaires : aucun test automatisé n'est présent dans le projet. L'architecture avec interfaces facilitera leur ajout.

Conclusion

Ce projet met en pratique plusieurs concepts fondamentaux du développement web avec .NET : la conception orientée objet à travers un modèle de données riche avec des relations bien définies, l'injection de dépendances avec le pattern Interface/Implémentation, l'accès aux données avec Entity Framework Core et ses migrations, et l'authentification et l'autorisation basées sur les rôles avec ASP.NET Core Identity. L'architecture en couches clairement séparées rend le code maintenable et extensible, et constitue une base solide pour des fonctionnalités futures comme l'intégration de prix en temps réel ou la génération de rapports PDF.